

Continnum：使用 KV 缓存生存时间实现高效、稳健的多轮 LLM 代理调度

Hanchen Li*

UC Berkeley

Berkeley, California, United States
lihanc@berkeley.edu

Runyuan He*

UC Berkeley

Berkeley, California, United States
runyuan@berkeley.edu

Qiuyang Mang*

UC Berkeley

Berkeley, California, United States
qmang@berkeley.edu

Qizheng Zhang

Stanford University

Stanford, California, United States
qizhengz@stanford.edu

Huanzhi Mao

UC Berkeley

Berkeley, California, United States
huanzhimao@berkeley.edu

Xiaokun Chen

Stanford University

Stanford, California, United States
xiaokunc@stanford.edu

Hangrui Zhou

Tsinghua University

Beijing, China
zhouhr23@mails.tsinghua.edu.cn

Huanchen Zhang

Tsinghua University

Beijing, China
huanchen@tsinghua.edu.cn

Alvin Cheung

UC Berkeley

Berkeley, California, United States
akcheung@cs.berkeley.edu

Joseph Gonzalez

UC Berkeley

Berkeley, California, United States
jegonzal@berkeley.edu

Ion Stoica

UC Berkeley

Berkeley, California, United States
istoica@berkeley.edu

摘要

KV 缓存管理是高效 LLM 推理的关键。为提高利用率，现有引擎在有新请求等待时驱逐已完成请求的 KV 缓存。该策略不适合交替执行 LLM 调用和工具的代理工作负载：工具造成的暂停阻碍跨轮 KV 复用。许多工具调用远短于多轮聊天中的人类响应时间，因此在工具执行期间保留缓存很有潜力，但仍有挑战。首先，需要同时考虑重计算或重载成本（启用卸载时），以及从 GPU 驱逐后增加的排队延迟。其次，工具耗时本身有方差，方法必须在可预测性有限时仍稳健。我们提出 Continnum，通过引入 KV 缓存生存时间机制，优化多轮代理的作业完成时间。对生成工具调用的请求，Continnum 根据重载成本和驱逐可能引起的排队延迟，选择性地将在 GPU 内存，并设置 TTL。TTL 到期后可自动驱逐缓存、释放内存，使系统在边界情形下仍稳健。结合程序级先到先服务，Continnum 保持多轮连续性并减少代理工作流延迟。在 SWE-Bench、BFCL、OpenHand 等真实代理，以及 Llama-3.1 8B/70B、Gemma-3 12B、GLM-4.5 355B 上评估表明，Continnum 将平均作业完成时间改善超过 8 倍，同时提高吞吐量。

PVLDB Reference Format:

Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Huanchen Zhang, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. Continnum：使用 KV 缓存生存时间实现高效、稳健的多轮 LLM 代理调度. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

*前三位作者对本工作贡献相同。

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 引言

KV 缓存管理对大型语言模型推理至关重要，影响输入处理（预填充）和输出生成（解码）阶段 [14, 28, 66]。其中关键组件是驱逐策略。理想情况下，应避免驱逐即将再次引用的 token。类似传统缓存，现有推理引擎假设解码完成后 KV 缓存重要性下降；为提高利用率，有新请求排队时就丢弃已完成请求缓存。我们称之为**轮末驱逐**。

轮末驱逐适合多轮聊天，却可能严重损害现代代理负载，特别是工具调用负载。这些应用在软件工程 [57]、计算机操作 [7] 和科学研究 [45] 等领域日益流行。其典型流程交替执行：(a) 推理出下一动作；(b) 调用外部工具。工具输出随后追加到请求上下文，触发新的推理步骤。工具往往比人类输入更快，例如 $\leq 2s$ ，因此需要改变轮末驱逐策略。

核心问题出现在代理从推理转入工具调用、请求 KV 缓存被驱逐后。如果该轮缓存已驱逐，则工具完成、下一推理开始时，引擎必须重算前缀（预填充），或从 CPU 重载（启用 CPU 卸载时 [14]）。反复预填充显著增加延迟并降低吞吐量。更关键的是，即使 CPU 卸载支持复用，驱逐仍导致**每轮排队延迟**：下一推理的缓存已离开 GPU，即便可从 CPU 重载，也必须先排队等待其他请求释放 GPU 内存。该延迟逐轮累积，增加每个代理程序的延迟，如图 1。它无法通过离线画像测量，需要新模型纳入影响。此外，工具调用本身可变，需要设置最大缓存保留时间，防止无限等待。但若在工具返回前刚好到期，此前等待将被浪费，因此必须谨慎设置保留时间，以适应负载。

既有工作未解决这些挑战。InferCept [2] 仅依据重载成本决定是否保留 KV，未建模逐轮累积的排队延迟，也缺乏处理可变工具耗时的稳健机制，使其难以实际部署。第 6 节将展示，InferCept 累积逐轮排队惩罚，导致次优表现。Autellix [39] 使用轮末驱逐，忽略多轮代理调度中的缓存保留重要性。Pie [20] 提供接口，但没有保留决策策略。Ayo [49]、Alto [46] 和 Parrot [34] 假设静态工作流，不适用于动态代理。

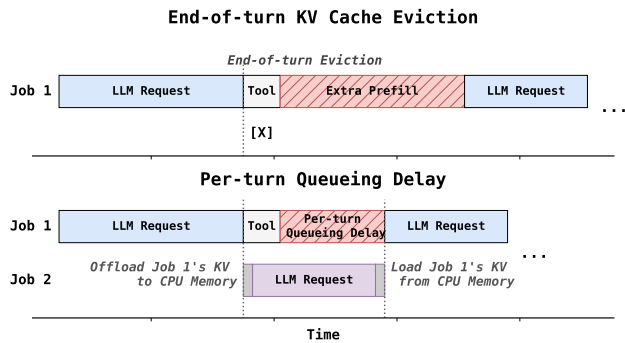


图 1: 既有代理服务系统的两类主要失败模式。红块表示次优调度和 KV 缓存管理的开销：即使启用 CPU 卸载，缓存驱逐后代理仍遭遇排队延迟。

为给出高效、稳健方案，我们提出 Continnum，使用 KV 缓存生存时间优化多轮代理作业完成时间。受既有缓存论文启发，Continnum 引入 TTL：请求完成后仍在 GPU 内保留缓存，覆盖原始轮末驱逐。对推理中生成工具调用的 LLM 请求，同时建模保留缓存减少的预填充／重载成本和每轮排队延迟。根据这两类因素及工具耗时分布得到潜在命中的收益后，与 TTL 期间占用 GPU 内存的成本比较，决定缓存在自动驱逐前保留多久。若工具在 TTL 内返回，下一请求可立即恢复，省去预填充及排队。当预测不准确、工具超时，TTL 到期驱逐能稳健纠正，避免严重内存压力或死锁。此外，结合程序级先到先服务，可改善请求顺序，简化复杂代理工作流调度。

我们在 vLLM 上以模块化方式实现 Continnum，便于维护或集成到其他引擎。工具调用处理器在请求进入或离开服务引擎时调用，识别工具、预测耗时，并根据吞吐量和请求顺序需求决定缓存固定超时。该设计对原始调度逻辑的改动很小，也支持未来扩展工具感知调度。

为评估性能，在函数调用 [41] 和编程代理 [33] 等真实负载上开展实验。在三种硬件／模型设置上，Continnum 将多轮代理延迟降低 1.12–3.66 倍，吞吐量提高 1.10–3.22 倍。我们还在 Company A 的内部测试平台¹评估，使真实 SWE 代理延迟最多降低 8.18 倍。我们将开放轨迹、代码和代理服务测试平台，以支持后续研究。

本文贡献如下：

- 识别代理服务中的关键 KV 缓存保留问题，说明需要更好的方案。
- 设计 Continnum，以 KV 缓存 TTL 实现高效、稳健服务，减少逐轮驱逐成本和每轮排队延迟。
- 在模拟和真实场景展示，相比既有方法，Continnum 的延迟和吞吐量均最多改善 8.18 倍。
- 论文发表后开放收集的代理推理轨迹、代码和服务测试平台。

2 背景

2.1 代理的 ReAct 范式

多数现代代理遵循 ReAct 循环 [61]：交替执行 LLM 解释上文并输出思考的推理步骤，以及调用外部工具的动作步骤。它

¹为双盲评审匿名化的一家推理初创公司。

表 1: Continnum 与代表性基线比较。

方法	保留 KV 缓存	纳入每轮排队延迟	限制保留时间
vLLM	✗	✗	✗
Autellix	✗	✗	✗
Pie	✓	✗	✗
InferCept	✓	✗	✗
Continnum	✓	✓	✓

已成为事实标准：Claude Code [8]、Cursor [16] 等编程代理因其清晰和性能而采用，LangChain [30]、LangGraph [31] 广泛提供此模式，GPT-OSS [3]、Kimi-K2 [1] 等开放权重模型更将工具调用能力直接融入基础模型。

一个重要趋势是将该循环扩展为长程、多轮迭代，在几十乃至几百轮中反复交替思考、工具调用和上下文更新。工具—代理—用户交互基准 τ -bench [60]、多轮工具增强交互 MINT [52]，以及多轮决策与工具使用 AgentBench [36] 都反映这一趋势。

2.2 现有方法的局限

既有工作无法处理这类新兴复杂负载，主要有三个原因：

固定工作流：一类工作调度具有**预定义**、**静态**计算图的代理工作流。Teola [49] 将应用分解为原语级数据流图并进行图优化；Alto [46] 关注分布式组件之间的流式、流水执行。

Parrot [34] 通过语义变量向 LLM 服务暴露应用级上下文，使引擎推断连续请求之间的数据依赖。Teola、Parrot、Alto 的共同局限是，假设静态或确定性定义的 DAG，**无法处理动态代理负载**，例如运行时依赖图持续演化的 ReAct 代理。这限制其优化实际多样代理的能力 [9, 33, 56]。

缺乏工具调用感知：Wadlom 等 [50] 从数据系统视角研究代理工作流的高效服务。Autellix [39] 引入程序级已获得服务 (PLAS)，优先累计服务时间较少的程序请求。Tempo [65] 针对聊天、代理、推理等多种请求满足 SLO，而本文专注多轮、可变工具调用代理。Apt-Serve [19] 联合优化混合缓存和调度，提高有效吞吐量，但面向独立请求而非动态多轮工作流。AlignedServe 探索前缀感知批处理 [10]。这些工作没有考虑工具耗时变化及其对缓存的影响，可能导致次优决策和更高延迟，后见第 3.2 节。

不足的 KV 缓存保留策略：部分工作观察到代理缓存复用挑战。Cache-Craft [4] 管理可复用块缓存以高效支持检索增强生成，但关注文档块，而非跨工具调用保留请求特有代理状态。InferCept [2] 引入“保留”操作，将工具调用之间的 KV 固定；但策略忽略请求多轮性质。轮间驱逐导致程序返回时每轮额外排队，累积后可能很大。忽略这一点，使它即使收益明显也不保留 GPU 缓存。其保留操作还固定不变，无法实时适应；实际工具耗时远超预测时，盲目保留可造成严重低效。

因此，其实际部署困难。Pie [20] 将生成循环分解为细粒度处理器，提供可编程服务，将控制权交给用户程序以自定义工具处理。但需要开发者为每个代理手工设计调度，没有实际适应动态工具延迟或多轮依赖的方法。

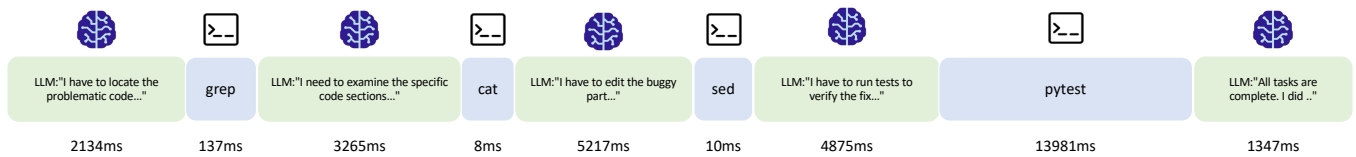


图 2: SWE 代理示例。代理通过逐步推理和中间工具调用解决软件工程缺陷。各工具耗时不同，打断 LLM 推理连续性。

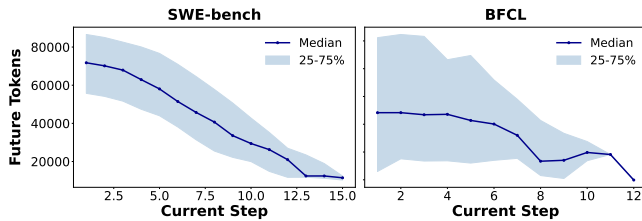


图 3: 第 6 节使用的 SWE-Bench 和 BFCL 代理负载特征。步数增加时，请求更接近完成。

表 2: 两个收集数据集的统计，数值格式为 (均值, 标准差)。

数据集	轮数	工具时间 (毫秒)	每程序 token 数
SWE-Bench	(10.9, 2.1)	(925, 3,550)	(70,126, 19,732)
BFCL v4	(6.3, 2.3)	(1,923, 2,133)	(93,256, 68,687)

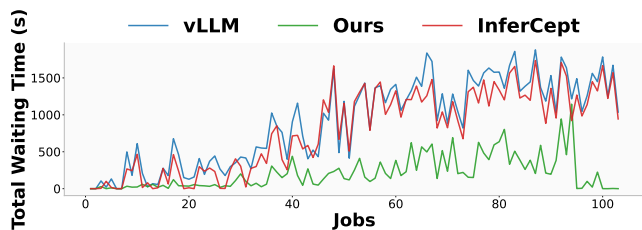


图 4: CPU 卸载下各程序的排队延迟。InferCept 保留决策忽略排队成本，因此被驱逐程序仍跨轮累积大量等待时间，尽管节省重载，其等待时间仍与原版 vLLM 相近。

3 动机

3.1 代理轨迹

首先分析现代代理负载特征。收集并分析 100 条 mini-swe-agent [33] 执行 SWE-Bench [23] 的轨迹，以及 100 条 BFCL V4 Web Search [40] 轨迹；两者均以 GPT-5 为基础模型。图 2 给出缩短的 SWE-Bench 示例，展示代理逐步解决软件工程任务。

有三点结论。第一，这类新代理程序具有较多轮，增加调度难度。第二，工具调用耗时分布多变，但不少很短。当前请求在生成短工具调用后被视为完成，下一请求却会在工具完成后很快到达并复用 KV。

第三，如图 3，程序接近完成时，两类负载的预期未来 token 总数都下降，意味着后期轮次预计完成时间更短。因此，优先较早到达程序（程序级 FCFS）或已执行较多轮的程序，可能较好近似理论最优但需要先知信息的最短剩余时间优先（SRTF）。然而涉及工具时，维持这种顺序并不简单，见后文 3.2。

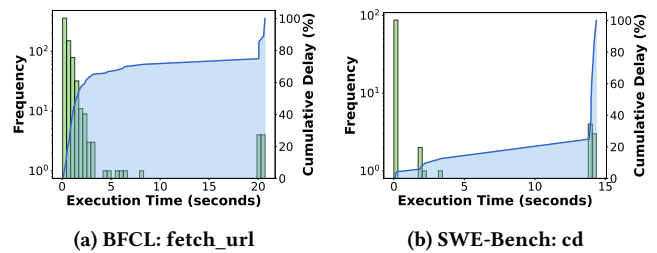


图 5: 函数执行时间可能极度长尾。最慢 10% 的 fetch_url 占总延迟的 52.5%，最慢 10% 的 cd 则占 94.1%。

3.2 代理负载的挑战

逐轮驱逐: 工具虽可能很短，引擎却将其视为 LLM 请求之间的同类间隔。vLLM、SGLang 在解码完成时驱逐 KV，隐含假设请求已完成。缓存一旦驱逐，引擎必须重做完整预填充，或启用卸载时从 DRAM 重载，增加延迟。多数系统无法高效处理。

图 1 展示：工具造成暂停，触发 KV 驱逐，返回后需要预填充或重载。因此，必须有考虑工具的缓存保留策略，避免这些开销。

每轮排队延迟: 多轮程序还带来既有工作忽视的调度挑战。当当前程序等待工具时，若调度器为提高吞吐量将 GPU 内存分给其他请求，当前程序 KV 就离开 GPU。工具返回并提交下一 LLM 请求后，它必须等待正在预填充/解码的其他请求释放内存。不论缓存是否存于 CPU DRAM，该等待都造成程序执行间隙。如图 1，除预填充/加载成本外，这也是工具引发的延迟，逐轮累积后显著增加程序延迟。它还打断连续性，使较早到达请求在较晚请求后执行。即使将返回请求设为等待队列最高优先级，它仍会被已在 GPU 上计算的请求阻塞。

既有保留策略未考虑每轮排队延迟。InferCept [2] 只有在 CPU 卸载成本超过工具执行期间估计 GPU 占用成本时才调用“保留”。它仅考虑下一轮的重载成本，完全忽略驱逐程序重新进入队列、排在活跃请求后的等待。LMCache [14] 等快速异步卸载降低重载成本，因此 InferCept 很少保留。但排队延迟与卸载速度无关：即使瞬间重载，返回请求仍需等待其他请求释放 GPU 内存。该成本在每一轮发生，总延迟随轮数成比例增长，正是代理负载的典型情形。

图 4 展示缺乏多轮考虑引起的退化。我们测量原版 vLLM 和 InferCept 各请求的总驱逐开销。横轴按到达顺序排列代理程序，纵轴是各作业总空泡时间，即请求执行前在等待队列中经历的总空闲时间。即使 InferCept 保留部分缓存、提高了 vLLM 吞吐量，空泡仍存在并增加延迟。

可变工具调用: 工具耗时高度可变时，当前保留策略也会失败。例如 InferCept 将 KV 固定在 GPU，直到工具完成、下一请求到达。这在稳定工具延迟下可行，但图 5 表明许多工具耗

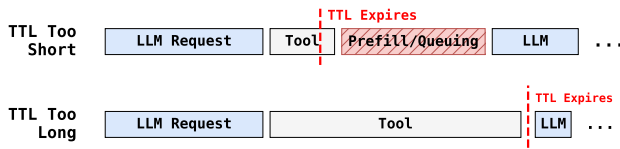


图 6: 需要合理设置存活时间, 在内存占用与预填充及每轮排队延迟之间取得平衡。

时变化很大。工具远超预计时, 固定缓存会长时间占用 GPU。数据库代理的外部工具更复杂, 也出现类似模式。它导致低效内存使用; 保留缓存占满 GPU 时甚至可能死锁。因此静态保留策略缺乏实际稳健性。

4 CONTINNUM 调度算法

针对既有工作的不足, 我们提出代理工作负载服务中的关键问题: 如何在多轮场景中高效而稳健地保留 KV 缓存?

理想的 KV 缓存保留策略应具备以下特征:

- 为工具调用后很快就会复用缓存的请求保留 KV 缓存, 尽量降低预填充或加载开销。
- 考虑代理程序的多轮连续性, 减少等待并维持程序顺序。
- 能够适应工具调用延迟的变化。

为实现稳健性保障, 我们借用传统系统中的存活时间 (Time-to-live, TTL) 机制: 为每个请求的 KV 缓存赋予一个 TTL 值, 规定其在 GPU 内存中最长保留多久。这样既能保留 KV 缓存, 又能避免耗时过长或失败的工具调用无限期占用 GPU 资源。

然而, 与静态保留操作相比, 为每份 KV 缓存设置适当的 TTL 更为困难。首先, TTL 不应过大。如图 6 所示, 若超时时间过长, 被固定保留的 KV 缓存会不必要地占用 GPU 内存, 阻塞其他请求并降低系统整体吞吐量。另一方面, 若保留时间过短, 工具调用尚未完成, KV 缓存便已被逐出; 即使此前已经付出了占用 GPU 的代价, 随后仍会出现昂贵的重计算或调度空隙。

因此必须谨慎设定 TTL。只有结合工具调用时长、预填充或加载成本, 以及程序连续性的度量, 才能在缓存复用收益与其他请求的系统吞吐需求之间取得平衡, 获得良好的性能。

4.1 效用模型

为了为请求的 KV 缓存选择有效的 TTL (单位为秒), Continnum 必须在潜在复用收益与占用成本之间选择最合适的平衡点。二者都以时间度量, 因为最终都会改变所有程序的作业完成延迟总和。数学上, 给定请求 r 和 TTL 值 τ , Continnum 估计将请求 r 的 KV 缓存固定保留 τ 时间的成本 $\text{Cost}(\tau, r)$ 与收益 $\text{Benefit}(r)$ 。为简化分析, $\text{Benefit}(r)$ 假设下一轮请求在 TTL 窗口内到达; TTL 先于工具调用返回而到期的情况在第 4.2 节处理。

成本估计。 固定保留请求 KV 缓存的成本, 来自占用本可用于服务其他请求的 GPU 内存所产生的机会成本:

$$\text{Cost}(\tau, r) = \frac{\text{MemUsage}(r)}{\mathcal{M}} \times \tau,$$

其中, $\text{MemUsage}(r)$ 是请求 r 的 KV 缓存所占 GPU 内存, $\mathcal{M}$ 是活跃请求的平均 GPU 内存占用, τ 为 TTL。

算法 1: Continnum 的调度算法

全局状态: 等待队列 Q ; TTL 映射 P (记录被固定保留的程序及其 TTL); 工具调用历史记录 S , 其中 $S[f]$ 表示工具 f 的调用记录

```

1 函数 OnRequestArrive(request r):
2    $Q \leftarrow Q \cup \{r\}$ ,  $id \leftarrow$  请求  $r$  的程序 ID;
3   如果  $id$  是此前出现过的程序则
4   |  $(f, t) \leftarrow$  来自  $r$  的工具调用信息;
5   |  $S[f] \leftarrow S[f] \cup \{t\}$ ;
6 函数 OnRequestFinish(request r):
7   如果  $r$  是所属程序的最后一个请求则
8   | 释放  $r$  使用的 KV 缓存;
9   否则
10  |  $f \leftarrow$  完成  $r$  后将调用的下一个工具;
11  |  $id \leftarrow$  请求  $r$  的程序 ID;
12  |  $P[id] \leftarrow \text{CalcTTL}(r, S[f])$ ;
13 函数 Schedule():
14  当  $Q$  非空 执行
15  | 对每个  $P.keys$  中的  $id$  执行
16  | | 如果 当前时间  $> P[id]$ , 且  $id \notin Q.programs$  则
17  | | | 释放  $id$  的最后一个请求使用的 KV 缓存;
18  | | |  $P \leftarrow P \setminus (id, P[id])$ ;
19  |  $r \leftarrow \text{argmax}_{r' \in Q} \text{CalcPriority}(r', P)$ ;
20  | 如果 内存无法容纳  $r$  则
21  | | 中断循环;
22  | 否则
23  | |  $Q \leftarrow Q \setminus \{r\}$ ;
24  | | 将  $r$  提交执行;
25  | |  $id \leftarrow$  请求  $r$  的程序 ID;
26  | | 如果  $id \in P.keys$  则  $P \leftarrow P \setminus (id, P[id])$ ;

```

表 3: Continnum 成本模型中请求 r 及其关联工具调用 f 的主要符号。

符号	含义
τ	存活时间 (TTL)
$\text{MemUsage}(r)$	r 占用的 GPU 内存
$\mathcal{M}$	已观察请求的平均内存占用
$\text{CacheMissCost}(r)$	重载 r 的成本
$\text{Prefill-Reload}(r)$	在 GPU 上重建 KV 缓存的时间
$\text{OutOfOrderCost}(r)$	请求 r 因乱序而产生的成本
η	工作负载的有记忆性系数
$\mathcal{T}$	平均等待时间
$\mathcal{P}(\tau, f)$	f 在 TTL 内完成的估计概率

比值 $\frac{\text{MemUsage}(r)}{\mathcal{M}}$ 表示固定保留 r 时会阻塞多少个平均大小的请求。换言之, 若保留 r 占用的内存相当于 k 个请求, 则保

留 r 会给大约 k 个其他请求各增加 τ 的延迟。我们假设, 需要保留 KV 缓存时, 等待队列中始终有足够多的请求, 因此确实会出现这种阻塞效应。

收益估计. 若请求在 TTL 内再次发出, 固定保留 KV 缓存便能使其避免为 r 所属程序重新加载或预填充 KV 缓存, 同时节省每轮的排队延迟:

$$\text{Benefit}(r) = \text{CacheMissCost}(r) + \text{OutOfOrderCost}(r)$$

这里, $\text{CacheMissCost}(r)$ 度量为请求 r 重载或预填充 KV 缓存的成本; $\text{OutOfOrderCost}(r)$ 度量该请求等待其他请求释放 GPU 内存所产生的预期排队延迟。收益取所避免的这两项成本之和。与 $\text{Cost}(\tau, r)$ 类似, $\text{CacheMissCost}(r)$ 可以通过以下两项度量: (1) 上下文重建开销 $\text{Prefill-Reload}(r)$; (2) 大约有多少个请求会受到这部分额外延迟的影响, 即 $\frac{\text{MemUsage}(r)}{\mathcal{M}}$ 。其正式定义为:

$$\text{CacheMissCost}(r) = \frac{\text{MemUsage}(r) \times \text{Prefill-Reload}(r)}{\mathcal{M}}$$

$\text{Prefill-Reload}(r)$ 表示预填充或重载的时间成本, 具体取决于是否启用 CPU 卸载; 该值通过第 5.2 节所述的快速非线性性能分析获得。

度量预期排队延迟: 如第 3.2 节所述, 保留 KV 缓存还能消除程序被逐出后重新返回队列时遭受的排队延迟, 即使 CPU 卸载已使重载本身很快。OutOfOrderCost 正是 InferCept [2] 等既有保留策略所缺少的关键项: 这些策略只考虑重载成本。建模这一项后, 即使重载很便宜, 只要节省的排队延迟大于 GPU 内存占用成本, Continnum 仍可合理地保留 KV 缓存。注意, 排队延迟收益与工作负载的有记忆性紧密相关, 即随着程序推进, 剩余步骤数是否会可预测地减少。

例如, 如果每个程序的请求总数服从几何分布, 则不论已服务多少个请求, 剩余请求数的期望都保持不变; 此时, 维持程序顺序不能使短作业优先完成, 因而固定保留缓存不能带来排队延迟收益。相反, 若每个程序发出的请求数固定, 则 TTL 可以通过近似最短作业优先消除排队成本。令 N 为程序的请求总数, k 为已经完成服务的请求数。我们定义如下有记忆性系数:

$$\eta = -\text{Corr}(k, N - k)$$

这个系数能够很好地刻画工作负载的有记忆程度: 完全无记忆时, k 与 $N - k$ 相互独立, 因此 $\eta = 0$; 完全有记忆时, 即所有程序的请求数均为同一固定值, 有 $\text{Corr}(k, N - k) = \text{Corr}(k, -k) = -1$, 故 $\eta = 1$ 。某些情况下, η 可能小于零, 例如轮数分布具有极长尾时; 这表示一种反记忆性模式: 程序越向前推进, 似乎反而揭示出更多剩余工作。我们没有观测到这种模式, 但 Continnum 的设计也考虑了此类极端工作负载: 此时更适合短暂服务各程序并频繁切换, 以适应轮数的长尾分布。

现在可以根据上述 η 定义 $\text{OutOfOrderCost}(r)$ 。当 $\eta = 1$ 时, 延迟恰为 r 所属程序重新返回等待队列时的等待时间。为对应这一情形, 我们记录工作负载中历史请求每单位上下文大小的平均等待时间 $\frac{\mathcal{T}}{\mathcal{M}}$, 其中 $\mathcal{T}$ 是此前请求的平均排队延迟。此时, 可用 $\frac{\mathcal{T}}{\mathcal{M}} \times \text{MemUsage}(r)$ 较好地度量延迟。这里引入 $\text{MemUsage}(r)$, 是因为上下文较大的请求更难调度: 它们必须等待足够的连续内存被释放。一般情况下, 乱序成本定义如下:

$$\text{OutOfOrderCost}(r) = \frac{\mathcal{T}}{\mathcal{M}} \times \text{MemUsage}(r) \times \eta.$$

4.2 设置 TTL

本节说明 Continnum 如何根据上述成本收益模型和工具调用历史信息, 为 KV 缓存设置 TTL。如算法 1 第 12 行所示, Continnum 选择最优 TTL τ^* , 使保留 KV 缓存的预期净收益最大化:

$$\tau^* = \arg\max_{\tau} \mathcal{P}(\tau, f) \times \text{Benefit}(r) - \text{Cost}(\tau, r), \quad (1)$$

其中, $\mathcal{P}(\tau, f)$ 估计工具调用 f 在时间 τ 内完成的概率。该式刻画将 r 的 KV 缓存保留 τ 时间所产生的预期净收益, 度量的总作业延迟。消去共有项 $\frac{\text{MemUsage}(r)}{\mathcal{M}}$ 后, 上式可转化为:

$$\arg\max_{\tau} \mathcal{P}(\tau, f) \times (\mathcal{T} \cdot \eta + \text{Prefill-Reload}(r)) - \tau, \quad (2)$$

这表明, 实现中只需额外计算 $\mathcal{T}$ 和 $\mathcal{P}(\tau, f)$ 。 $\mathcal{T}$ 可由被逐出请求所经历的排队延迟的滑动窗口平均值估计。由于无法完全预测下一次工具调用的时长, 我们用历史工具调用记录 $S[f]$ 得到的经验累积分布函数 (CDF) 估计 $\mathcal{P}(\tau, f)$ 。具体计算方式为:

$$\mathcal{P}(\tau, f) = \frac{1}{|S[f]|} \cdot \sum_{t \in S[f]} \mathbb{I}[t \leq \tau]$$

, 其中, $\mathbb{I}[\cdot]$ 为指示函数。最后, 将 $S[f]$ 中所有不同的工具调用时长作为候选值, 并包含 $\tau = 0$, 逐一代入式 (2), 选择预期收益最大的值。

冷启动处理. 当 $S[f]$ 中的历史记录很少时, 经验 CDF 的估计可能不可靠。此时先尝试利用全局工具调用信息估计 $\mathcal{P}(\tau, f_{\text{any}})$, 其计算式为 $\sum_{t \in S} \mathbb{I}[t \leq \tau] / |S|$ 。

此外, 在引擎刚开始提供服务时, 甚至全局记录也可能不可靠。为此, 我们设计了 Continnum 的最小版本, 采用固定 TTL 阈值 T_{default} 。该阈值仍由同一成本模型推导, 但假设工具调用时长服从均值为 1 的指数分布, 即 $\text{ToolCallDuration} \sim \text{Exp}(1)$; 同时假设工作负载完全有记忆, 即 $\eta = 1$ 。在此情形下, 将 T_{default} 设为最优的 τ^* 。

实践中设置阈值 M , 决定使用固定 TTL、全局记录, 还是上述基于 $S[f]$ 的细粒度估计。具体而言, $|S| \leq K$ 时使用 T_{default} ; 否则, 若 $|S[f]| \leq K$ 则使用全局记录, 其他情况采用细粒度 TTL。实现中取 $K = 100$, 并将 $\mathcal{T}$ 初始化为零。此外, 代理通常会在投产前结合工具进行后训练 [12, 13, 38], 因此用户也可在训练期间获取这些成本模型统计量。

4.3 调度优先级

为了使调度与 TTL 算法兼容, 需要重新定义推理引擎中的请求优先级。Continnum 引入感知 TTL 的优先级, 提升 TTL 内仍被固定保留的请求, 以维持连续性, 同时保留程序级先来先服务 (FCFS) 顺序。具体而言, 调度器为等待队列 Q 中的每个请求 r 赋予多键优先级元组, 按以下标准依次排序:

- **被抢占状态:** 与原引擎一致, 因运行队列竞争而被抢占的请求优先于未被抢占的请求。
- **TTL 状态:** 在其余请求中, TTL 窗口内缓存仍被保留的请求优先于未固定保留缓存的请求。
- **程序级到达顺序:** 最后, 各类内部按所属程序的到达时间排序, 以维持 FCFS 公平性。

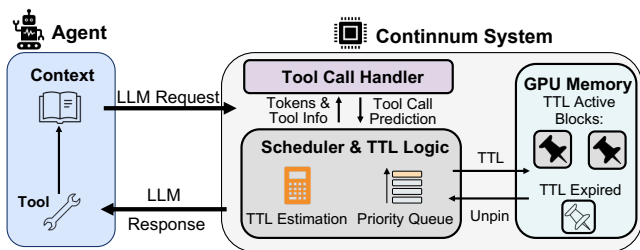


图 7: Continnum 系统概览。

5 CONTINNUM 系统设计

Continnum 的设计目标是构建模块化架构，使核心推理引擎的调度循环只需极少修改。

在客户端，为每个推理请求附加程序标识符（`program_id`），以便系统识别多轮代理程序，并跨步骤分析工具调用。

请求抵达服务引擎后，进入原有调度循环。Continnum 添加了一个轻量的工具调用处理器，在请求到达和完成时调用。处理器从 LLM 输出中解析工具调用，利用同一 `program_id` 的请求间隔跟踪各工具的延迟，并向调度器返回 TTL。调度器依据这一提示固定保留请求的 KV 缓存，供下一步潜在复用；TTL 到期或程序终止后，再解除固定。

5.1 工具调用处理器

工具调用处理器是一个独立类，由主调度器在请求到达后或完成时调用。这种解耦结构使工具处理逻辑独立于核心调度循环，便于将来扩展解析器或感知工具的策略。

识别工具调用：调度器完成请求后，将响应交给工具调用处理器，由后者判断响应是否包含工具调用。处理器按函数调用模式解析消息，因为 LLM 输出经常采用标准化工具调用结构，例如以下 OpenAI 模式：

```
{
  "id": "fc_0",
  "call_id": "call_0",
  "type": "function_call",
  "name": "get_weather",
  "arguments": {"location": "Paris"}
}
```

对于这一示例模式，处理器检查每个返回消息块的 `type`；若它表示函数或工具调用，则提取 `name`，作为工具调用类型。在 SWE-Bench 中，包含函数调用的每个 LLM 响应都保证恰好包含一次 `bash` 函数调用。我们提取 `bash` 块内的字符串，再取其首个词作为工具调用名称。对于不同 LLM 的其他函数调用格式 [35, 44]，也可以按相同方式扩展解析器，增加针对各模式的提取逻辑。

记录工具完成时间：对于程序 ID 为 p 的程序中的每个 LLM 请求 i ，当调度器记录一项带有工具调用输出的已完成请求时，处理器记录服务端完成时间戳 $t_{\text{finish}}^{p,i}$ 及工具调用名称。同一 p 的下一个请求 $i+1$ 到达时，观察其服务端到达时间戳 $t_{\text{arrive}}^{p,i+1}$ ，并计算请求间隔 $t_{\text{arrive}}^{p,i+1} - t_{\text{finish}}^{p,i}$ 。将这一间隔记录为本次工具调用的执行时间，供以后计算 TTL。

5.2 调度器中的高效 TTL 固定保留

工具调用处理器给出 TTL 后，调度器需要执行固定保留操作。

固定保留请求：若步骤没有被标识为最后一步，例如已解析出其中含有工具调用，调度器便调用处理器获取 TTL τ^* ；若其非零，则调用 `pin_request(request,  $\tau^*$ )`。该操作将请求及其到期时间 `current_timestamp +  $\tau^*$` 作为一对记录写入字典 `pinned_requests`，并有意跳过释放请求 KV 块的操作。`pinned_requests` 还会传入等待队列，以优先调度同一程序的下一轮请求。

解除请求固定：每个调度步骤开始时，调度器执行 `unpin_requests()`，扫描 `pinned_requests`，对 TTL 已到期且其 `program_id` 当前不在等待队列中的条目解除固定。这可以避免后续请求已到推理引擎、但尚未获调度时缓存被过早逐出。此外，程序最后一步完成时，调度器会主动解除具有相同 `program_id` 的所有剩余固定，因为短期内不再预期复用其 KV 缓存。

防止死锁：被固定保留的请求可能不断积累。当它们占满所有 GPU 内存时，可能出现死锁。由于同一程序的下一轮请求仍在等待队列中时，已固定的请求会继续保留，整个调度循环可能因空间不足而停滞，无法调度任何新请求。因此，需要在这种死锁发生时解除部分请求的固定。在 Continnum 中，调度逻辑因空间竞争无法安排新请求时，会检查 `pinned_requests` 是否存在固定请求；若存在，则反复从 `pinned_requests` 中选择程序到达时间最晚的请求，解除固定并释放空间，直到队首请求可以执行。选中的请求从其队列中移除，其 KV 缓存被释放，并按需重新入队，使后续分配可以继续。即使固定请求很多，这也能避免死锁。

离线性能分析：为了按第 4.1 节的需要，根据上下文大小预测预填充和重载时间（`Prefill-Reload(r)`），我们为每种硬件与模型组合做离线性能分析，供在线估计使用。分析有两个目的：（1）CPU 卸载情形下的 GPU-CPU 带宽，用平均 CPU 卸载吞吐量度量；（2）预填充成本与上下文长度的关系曲线。为此，对 $\{1000, 2000, 4000, \dots, \text{max_context_length}\}$ 等分块大小执行预填充，再用数据拟合二次曲线。确实可能有部分请求页面仍在 GPU 内存中，无需重计算；不过，内存竞争时这些剩余页面通常很少，因此用完整预填充时间近似只会产生很小误差。每种硬件与模型组合的分析耗时不足十分钟。

5.3 实现

我们在 vLLM 上用约一千行 Python 实现了 Continnum。除在调度器类中加入上述固定操作外，还在 vLLM 原调度器中使用工具调用处理器的三个函数：

- `func_call_finish(tool, timestamp)`: 请求完成且被解析为含有工具调用时，该函数通知处理器记录工具调用的开始时间。
- `update_tool_call_time(program_id, timestamp)`: 新请求到达意味着前一请求的工具调用已经完成，因此记录这一时间。
- `set_up_ttl(request, tool)`: 根据既往工具调用信息及系统配置，为这个已完成请求计算供调度器使用的最佳 TTL。

6 评估

评估得出以下主要结论：

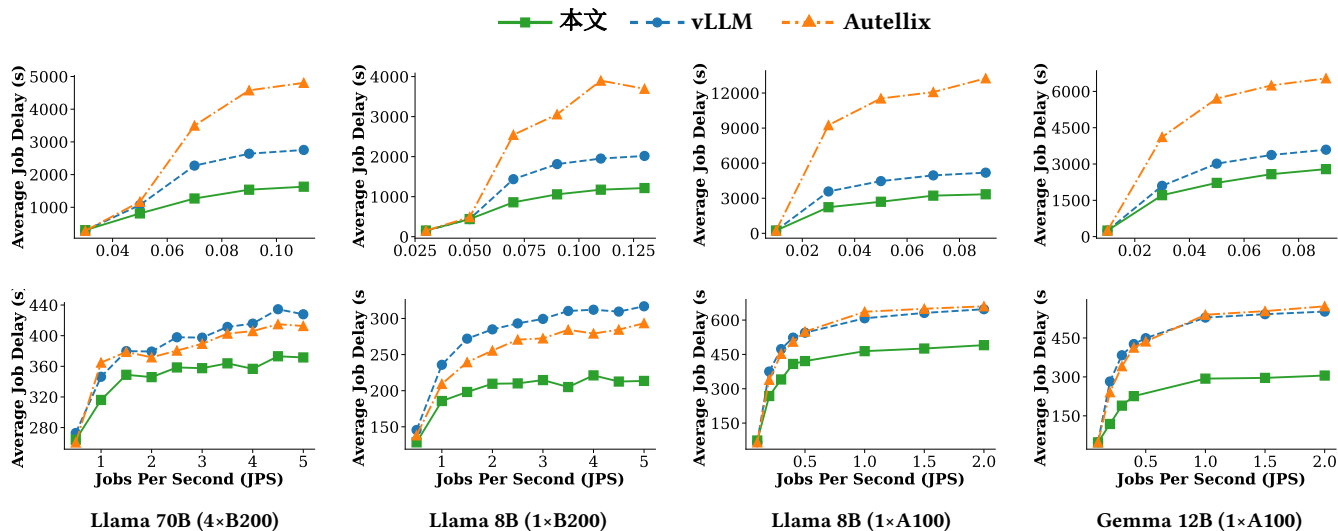


图 8: Continnum 在不同模型规模、硬件配置及数据集上均优于基线调度器（上排：SWE-bench；下排：BFCL）。

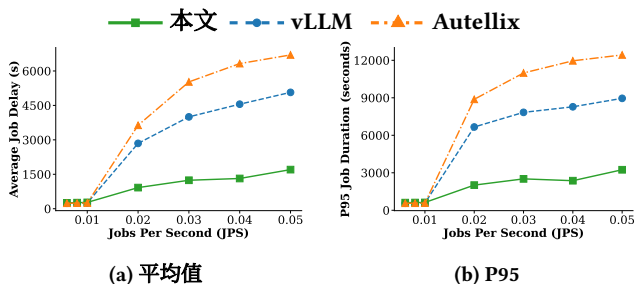


图 9: 在 H100 上使用 Llama-8B 运行 OpenHands 时, Continnum 的平均延迟和 P95 延迟均最优。

- **降低延迟:** Continnum 通过智能固定保留 KV 缓存, 相比基线调度器显著降低延迟。
- **稳健性:** 在不同轮数和卸载场景下, Continnum 均优于基线。
- **开箱即用:** Continnum 能在不降低质量的情况下加快真实代理运行。

6.1 实验设置

模型与硬件: 使用 Llama-3.1-8B、Llama-3.1-70B 和 Gemma-3-12B 评估 Continnum。硬件包括 Runpod 的 A100-SXM GPU、AWS 及 Company A 的 H100, 以及本地服务器的 B200 GPU。

数据集: 除图 12 的真实 SWE-Bench 实验外, 我们在两类通过运行 GPT-5 收集的工作负载上评估², 代理程序的到达模式采用泊松分布:

- SWE-Bench [24]: 在 SWE-Bench 上运行 mini-swe-agent [33]³, 只保留上下文窗口内的请求。

²使用 GPT-5 是为了利用其更强的模型能力, 使生成的工作流大多正确。较小的基础模型常常无法完成任务。

³SWE-bench 官方代理, 截至 4 月 13 日在排行榜上位列第 5。

- Berkeley Function Calling Leaderboard [40]: 使用 BFCL V4 最新版本的 Web Search 类别, 其中代理通过网页浏览工具回答问题。将工作负载缩至原来的 0.4, 以便在 llama-3.1 的 128k 词元上下文窗口内至少容纳 100 个请求。
- OpenHand [51]: OpenHands 是常用的开源编程代理。运行其官方仓库中面向 Go 语言的 multi-SWE-bench [64] 示例。

主要基线:

- **原生 vLLM:** 使用稳定版本 vllm 0.10.2 的默认设置, 启用分块, 大小为 2048。
- **CPU DRAM 卸载:** 使用 vllm 0.10.2 和 LMCACHE 0.3.7 [14]。A100 每 GPU 的卸载 DRAM 配为 100GB; B200 和 H100 每 GPU 配为 200GB。也为以下算法启用同样的卸载。
- **Autellix:** 在 vllm 上实现 Autellix [39] 的 PLAS 算法; 启用 LMCACHE, 将 Autellix 扩展到 CPU 卸载场景, 称为 Autellix+。
- **InferCept:** 在 vllm + lmcache 上实现 InferCept [2] 的选择性保留、交换或逐出算法。LMCACHE 的 CPU 卸载非阻塞, 优于原始 InferCept, 因此相应更新了成本估计。
- **分布式推理:** 真实代理实验还比较其他开源方案, 包括采用原生缓存感知路由的 SGLang 0.5.5.post3 [47], 以及配置为 1P1D、将预填充和解码分离的 Nvidia Dynamo 0.7.0.post1 [6]。

6.2 端到端实验

我们对 SWE-Bench、BFCL 和 OpenHands 工作负载进行轨迹回放实验。图 8、图 10 和图 9 展示了 Continnum 的端到端改进。BFCL 与 SWE-Bench 上, 平均响应时间和吞吐量均显著改善。例如, 使用 Llama-3.1-8B 时, 相比原生 vLLM 基线, Continnum 将平均响应时间最多缩短至约二分之一。不同模型规模和硬件配置下都有稳定收益, 说明方法在多种场景中有效。虽然 Autellix 在 BFCL 上优于基线, 但在 SWE-Bench 上表现较差, 原因是它错误地假设请求执行越久, 预期完成时间越长。注意, 本文每秒作业数低于此前 LLM 服务论文报告的数值, 因为代

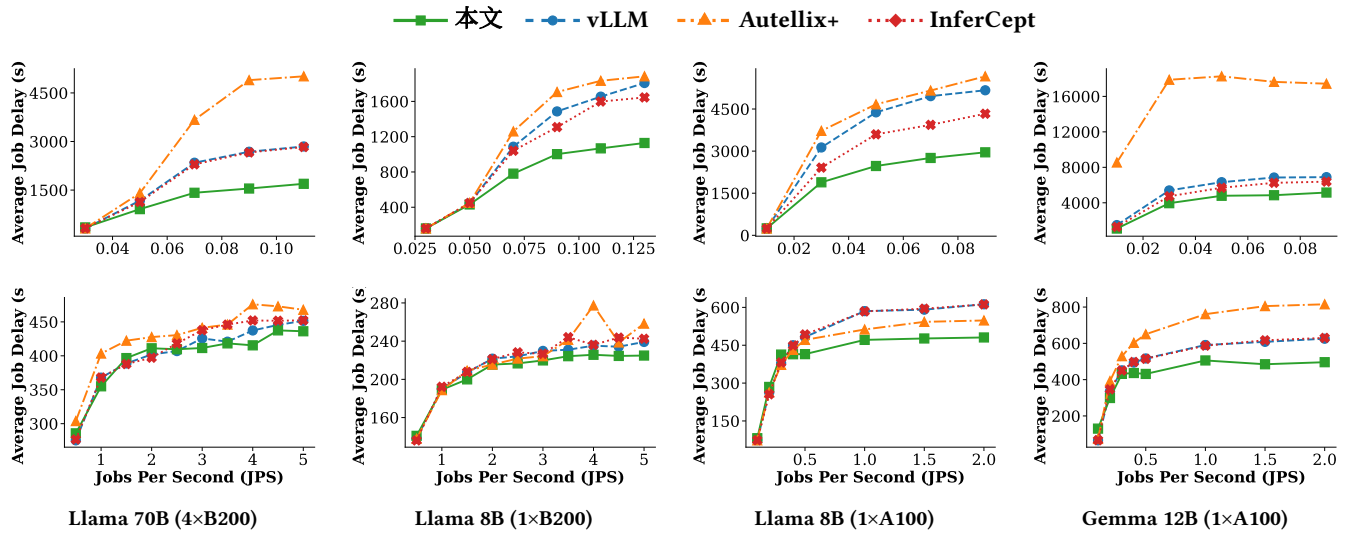


图 10: 启用 DRAM 卸载时, Continuum 仍能稳定改善性能。通过同时考虑工具调用和多轮执行, 它优于 InferCept 等具有智能 DRAM 卸载逻辑的系统 (上排: SWE-Bench; 下排: BFCL)。

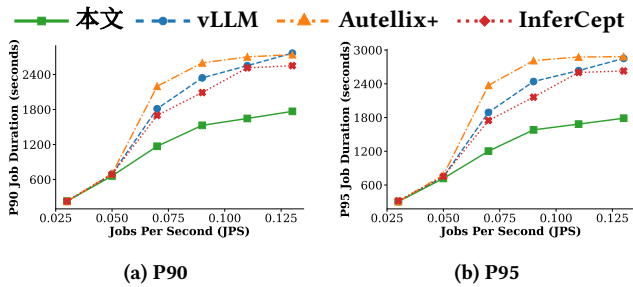


图 11: 用 Llama-8B 模型运行 SWE Bench 轨迹时, Continuum 获得更低的 P90 和 P95 延迟。

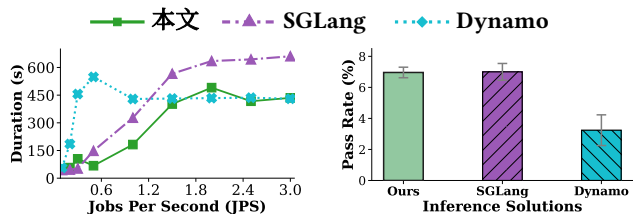


图 12: 在分布式环境中运行真实 SWE 代理时, Continuum 在保持通过率的前提下改善延迟。

理工作负载复杂得多, 往往涉及十余次 LLM 推理请求, 计算负担更大。

评估还扩展到其他实用代理。如图 9 所示, 在 AWS 的单张 H100 上使用 Llama 8B 运行 OpenHands 时, 本文方法获得更低延迟。由于平均轮数更高, 基线在多轮情况下退化更明显, 因此我们的改善也更显著。

此外, Continuum 始终优于 CPU 卸载基线; 另一方面, 启用 CPU 卸载后, PLAS 相对基线的收益有所减弱。这说明 Continuum 在减少调度空隙方面的稳健改进, 与 DRAM 卸载技术相互独立。

图 11 表明, Continuum 能减少每轮排队延迟, 因此相比基线获得更低的 P90 和 P95 延迟。各数据点的设置为在单张 B200 上运行 Llama-8B, 每 GPU 配置 200GB CPU 卸载空间。

分布式环境中的真实 SWE 代理: 为全面评估 Continuum 在真实大规模部署场景中的性能, 我们在 Company A 内部 H100 测试平台上, 用真实 SWE 代理执行 SWE-Bench-Verified 的 500 个任务。在 SWE-Bench 客户端环境中加入作业分发器, 使代理按泊松分布到达。Continuum 使用简单的会话感知路由, 并与其他分布式推理方案比较。我们测量每个作业的完成时间, 并在生成结束后, 收集各代理程序结果在 SWE-bench 上的通过率。

如图 12 所示, 通过率相同时, Continuum 的平均延迟始终优于基线。实际上, Continuum 的通过率还更高, 原因是 SWE-Bench 为环境 Docker 容器设置时间限制, 以免执行挂起; 基线运行超过 15 分钟便被抢占, 并计为失败。这验证了 Continuum 在真实生产环境中的可用性。

6.3 敏感性分析

改变推理引擎配置: 为说明 Continuum 能适应不同推理引擎配置, 我们在多种配置下评估。在图 13 中, 固定每秒作业数为 0.13, 改变最大批大小, 将 Continuum 与各基线比较。可以看到, 其改善在不同批大小下保持稳定。此外, 在图 13 中将分块大小从 256 改为 4096, 同样观察到类似收益。这说明方法对不同推理引擎配置具有稳健性。

轮数的扩展规律: 图 14 评估多轮场景下的稳健性。在 SWE-Bench 上将轨迹重复 1x 至 5x, 同时反向缩短词元长度, 模拟轮数更多、但总词元数仍可容纳在上下文窗口内的场景。每秒作业数为 0.13, DRAM 卸载容量为 200GB。结果表明, 随着轮数增加, 基线方法逐渐退化: 更多轮次意味着更多工具调用和

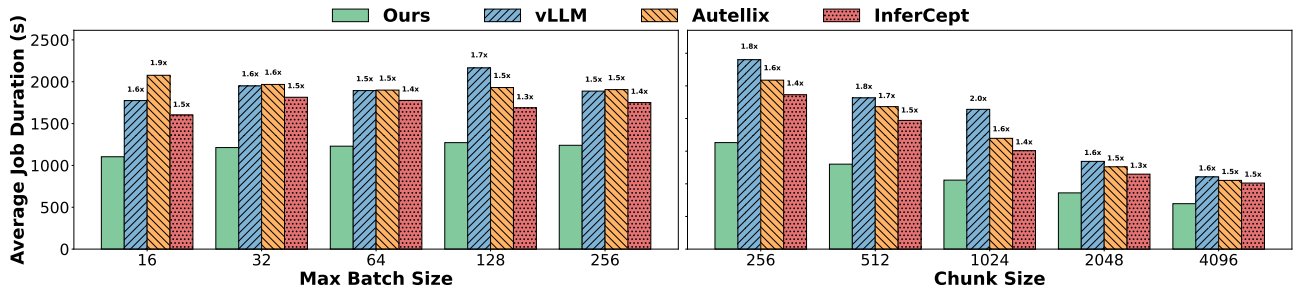


图 13: 在不同最大批大小及分块大小配置下, Continnum 均改善延迟。

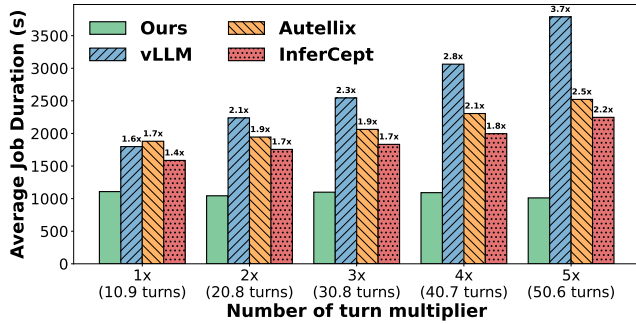


图 14: 轮数增多时, Continnum 的相对改善幅度更大, 其延迟仍保持稳定。

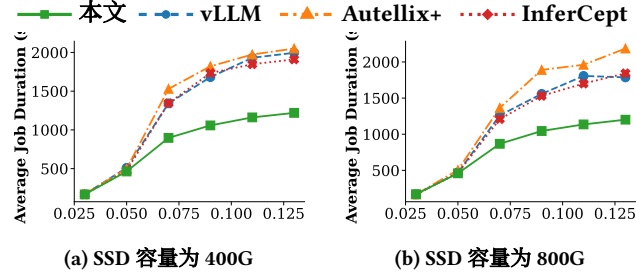


图 15: 在 CPU 卸载基础上进一步使用 SSD 卸载设备时, Continnum 仍能降低延迟。

更长的整体执行时间, 加剧传统方法面临的调度挑战。相比之下, 本文方法保持稳定的低延迟, 说明它适合复杂且轮数较多的代理交互。

SSD 卸载: 与 CPU 卸载类似, SSD 卸载提供更大空间, 但加载更慢。在 B200 上使用 llama-8B 和 SWE-bench 工作负载, 借助 LMCACHE 将 SSD 存储层扩展到 CPU 卸载之外来评估 Continnum。如图 15 所示, 同时使用不同容量的磁盘时, Continnum 仍能相对基线稳定降低平均延迟。

6.4 消融研究与微基准测试

消融研究: 我们通过消融分析成本建模对 Continnum 整体性能的影响。在图 16 中, 将 Continnum 与只应用部分优化的基线比较。程序级 FCFS 将 vLLM 原有的请求级 FCFS 改为按程序

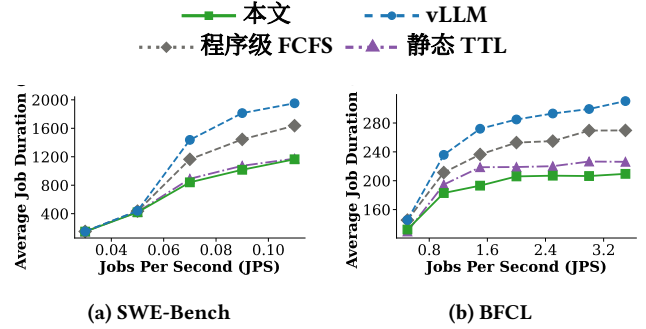


图 16: 各个设计对 Continnum 的贡献。程序级 FCFS 优先服务程序到达时间更早的请求, 而非单个请求到达更早者。静态 TTL 使用冷启动处理机制算出的固定 TTL 阈值。

表 4: 不同 DRAM 卸载设置下的调度延迟比较; Continnum 的额外开销很小。

系统	不启用 CPU 卸载	启用 CPU 卸载
vLLM	0.95 ms	2.33 ms
Autellix	0.82 ms	2.18 ms
InferCept	不适用	2.25 ms
本文	0.96 ms	2.30 ms

到达时间确定优先级; 静态 TTL 在程序级 FCFS 基础上, 使用冷启动处理中估计的固定 TTL 阈值。结果显示, Continnum 的各项设计逐步改善性能。

调度器开销: 如表 4 所示, 本文方法相比基线只引入很小的调度开销, 处于个位数毫秒量级, 相对于 LLM 推理的 GPU 执行时间可忽略。调度策略带来的显著端到端收益远大于这部分轻微增加的调度延迟。

强化学习应用: 我们还为 Continnum 在强化学习中的潜在用途进行了微基准测试。在 Multi-SWE bench [64] 上, 使用 GLM-4.5-fp8 和 OpenHands 代理生成训练 rollout, 硬件为一个 8 张 H100 的节点。按同期强化学习工作 ThunderAgent [26] 原论文报告的每分钟推理步骤数进行比较。如表 5 所示, Continnum 的单节点 rollout 吞吐量更高。

表 5: Continnum 的 OpenHands rollout 性能优于同期工作。

	vLLM	ThunderAgent	Continnum
吞吐量（每分钟步骤数）	93.4	114.8	144.9

7 相关工作

LLM 推理系统: 许多研究改进 LLM 推理。vLLM [28] 和 SGLang [66] 通过分页注意力和优化内核达到先进表现。除提高 GPU 执行速度的内核优化 [17, 62, 68], 还提出连续批处理 [63]、分块预填充 [5]、跳级加入式多级调度 [53] 等资源管理优化, 许多已移植至引擎。既有工作还研究高效卸载到 CPU DRAM 和磁盘 [14, 18, 37, 55, 59]。分布式推理使用会话感知路由 [29, 48]、KV 缓存感知路由 [54] 及预填充—解码分离 [67]。Continnum 在此基础上将推理扩展到长程多轮代理负载, 改善不同请求竞争时的资源管理。

计算机系统中的生存时间机制: TTL 是长期使用的系统抽象, 广泛用于 DNS 解析器、分布式缓存、CDN 边缘节点及一致性协议, 限制陈旧程度并防止无限保留资源 [11, 15, 21, 22, 25, 27, 32, 42, 43, 58]。TTL 作为粗粒度有效窗口, 在不可预测更新或获取延迟下平衡新鲜度、负载和稳健性。我们将这条技术线扩展到 LLM 引擎内部细粒度资源管理。传统 TTL 条目独立, 正确性约束主要是语义性的; 而 KV 缓存紧密关联 GPU 内存压力、预填充成本和调度公平性。

据我们所知, Continnum 是首个根据预测工具耗时、调度延迟传播及负载模式, 用 TTL 调节 LLM KV 缓存的系统。

超越 ReAct 代理的通用性: 当前设计针对交替调用工具的 ReAct 代理: 每个 LLM 步骤返回明确工具调用, 下一步前存在间隔。Continnum 自然支持并行工具调用, 因为仍遵循顺序“推理 → 工具 → 推理”节奏。某些新框架可能包含非线性控制流: 推测分支、异步多代理协作及上下文折叠。这类负载目前多为实验性, 尚待真实生产验证, 其推理模式可能违背顺序流程, 需要未来改变。扩展支持此类负载是重要方向。

8 结论

代理工作负载频繁调用工具、步骤间延迟变化显著, 并且需要维持多轮连续性, 因而给 LLM 服务系统带来新的调度挑战。本文提出 KV 缓存保留与调度系统 Continnum, 利用存活时间机制平衡缓存复用收益和占用 GPU 内存的阻塞成本。将基于 TTL 的固定保留与程序级 FCFS 结合后, Continnum 减少不必要的预填充, 缓解每轮排队延迟, 并稳健适应不可预测的工具调用时长。基于 vLLM 的实现, 在不同模型规模、硬件配置和真实代理工作负载上均改善了端到端作业完成时间。Continnum 表明, 有原则且感知工具 KV 管理对高效多轮代理服务十分关键。我们希望它为未来系统将代理工作负载深入集成到 LLM 推理引擎中奠定基础。

9 数据可用性

实现与实验材料可获取于: <https://github.com/Hanchenli/vllm-continnum>

图中英文与算法标识的中文对照

本译本保留原图的曲线、数值和英文标识, 以便与原文逐一对应。图内英文的完整中文含义如下:

- SWE 代理轨迹: “I have to locate the problematic code” 为“我必须定位有问题的代码”; “I need to examine the specific code sections” 为“我需要检查具体代码段”; “I have to edit the buggy part” 为“我必须修改存在缺陷的部分”; “I have to run tests to verify the fix” 为“我必须运行测试来验证修复”; “All tasks are complete. I did...” 为“所有任务已完成。我已完成……”。grep、cat、sed、pytest 是工具名称; ms 为毫秒。
- 逐轮逐出示意图: End-of-turn Eviction / End-of-turn KV Cache Eviction 为“轮次结束时逐出/轮次结束时逐出 KV 缓存”; Job 为“作业”; LLM Request 为“LLM 请求”; Tool 为“工具”; Extra Prefill 为“额外预填充”; Time 为“时间”; Offload Job 1’s KV to CPU Memory 为“将作业 1 的 KV 缓存卸载到 CPU 内存”; Load Job 1’s KV from CPU Memory 为“从 CPU 内存加载作业 1 的 KV 缓存”; Per-turn Queueing Delay 为“每轮排队延迟”。
- 未来需求与工具耗时图: Current Step 为“当前步骤”; Future Tokens 为“未来词元数”; Median 为“中位数”; 25–75% 为“第 25 至第 75 百分位区间”; Jobs 为“作业”; Total Waiting Time (s) 为“总等待时间 (秒)”; Execution Time (seconds) 为“执行时间 (秒)”; Frequency 为“频次”; Cumulative Delay (%) 为“累计延迟占比 (百分比)”。fetch_url 和 cd 保留为工具名。
- TTL 示意图: TTL Too Short / TTL Too Long 为“TTL 过短/ TTL 过长”; TTL Expires 为“TTL 到期”; Prefill/Queueing 为“预填充/排队”。
- 系统图: Scheduler & TTL Logic 为“调度器与 TTL 逻辑”; Agent 为“代理”; LLM Response 为“LLM 响应”; Tool Call Handler 为“工具调用处理器”; Context 为“上下文”; GPU Memory 为“GPU 内存”; TTL Active Blocks 为“TTL 有效的缓存块”; TTL Expired 为“TTL 已到期”; Tool Call Prediction 为“工具调用预测”; Tokens & Tool Info 为“词元和工具信息”; TTL Estimation 为“TTL 估计”; Priority Queue 为“优先队列”; TTL Unpin 为“解除 TTL 固定”; Continnum System 为“Continnum 系统”。
- 所有实验曲线: Jobs Per Second (JPS) 为“每秒作业数”; Average Job Delay (s) / Average Job Duration (s) 为“平均作业延迟/平均作业时长 (秒)”; P90 / P95 Job Duration (seconds) 为“作业时长的第 90 / 95 百分位 (秒)”; Duration (s) 为“时长 (秒)”; Inference Solutions 为“推理方案”; Pass Rate (%) 为“通过率 (百分比)”; Max Batch Size 为“最大批大小”; Chunk Size 为“分块大小”; Number of turn multiplier 为“轮数倍增系数”; turns 为“轮”; Ours 为“本文”; Program FCFS 为“程序级先来先服务”; Static TTL 为“静态 TTL”。图上的 x 表示倍数; SWE-bench、BFCL、模型和基线名称保留原名。
- 算法标识: OnRequestArrive / OnRequestFinish 为“请求到达时/请求完成时”; Schedule 为“执行调度”; CalcTTL 为“计算 TTL”; CalcPriority 为“计算优先级”; request 为“请求”; keys 为“键集合”; programs 为“程序集合”。JSON 示例的 get_weather 为获取天气的函数标识, location 为地点字段, Paris 为巴黎; 可执行字段名、函数名与变量名保留原样。

译者说明

本文全文由当前对话模型直接翻译, 未调用本地语言模型或外部翻译服务。正文、算法、表格、图注及有效源码中的全部章节均已纳入; 本次源码没有单列附录。数学表达式、引用、程序接口及机构身份信息按原文保留。

arXiv 标题使用 Continnum, 但源码的系统名宏和部分图表写作 Continnum。译文保留源码的这一拼写, 输出文件按 arXiv 标题命名。冷启动段先将阈值记为 M , 随后使用 K , 实现取 $K = 100$; 此处保留原符号。算法将 $P[id]$ 描述为 TTL, 但到期判断直接与当前时间比较; 系统实现进一步说明字典保存的是当前时间加 TTL 得到的到期时间。

效用模型将内存占用折算为受阻请求的累计时间成本, 假设需要缓存保留时等待队列中有足够请求; 它是用于 TTL 选择的估计模型。关于大上下文等待“连续内存”的表述也按原文保留, 不应自行扩展为所有采用分页 KV 缓存的引擎均要求物理连续内存。几何分布的无记

忆性说明依赖原文对轮数分布的假设，程序级 FCFS 在一般随机程序中并不因此获得最短作业优先的最优性保证。

数据集称“两类”采集工作负载，但随后也列出 OpenHands，译文保留原文结构。多数端到端曲线是 GPT-5 生成轨迹的回放，而真实执行测试使用 SWE-Bench-Verified 的 500 个任务。真实测试中的较高通过率部分来自较少触发 15 分钟超时，不能直接解释为模型推理质量改善。轮数扩展实验通过重复轨迹并反向缩短词元长度构造，不能直接视为自然产生的长轮次工作负载。摘要与贡献段给出的最大倍数应结合各实验条件理解，不能套用于所有模型、负载或吞吐指标。

参考文献

- [1] 2025. Kimi K2 Tech Blog. <https://kimi-k2.org/blog>. Accessed: 2025-12-08.
- [2] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiyang Zhang. 2024. INFERENCE: Efficient Intercept Support for Augmented Large Language Model Inference. In *Forty-first International Conference on Machine Learning*. Vienna, Austria.
- [3] Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K Arora, Yu Bai, Bowen Baker, Haiming Bao, et al. 2025. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925* (2025).
- [4] Shubham Agarwal, Sai Sundaresan, Subrata Mitra, Debabrata Mahapatra, Archit Gupta, Rounak Sharma, Nirmal Joshua Kapu, Tong Yu, and Shiv Saini. 2025. Cache-craft: Managing chunk-caches for efficient retrieval-augmented generation. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–28.
- [5] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 117–134.
- [6] ai dynamo. [n.d.]. Dynamo. <https://github.com/ai-dynamo/dynamo>. Accessed: 2025-12-09.
- [7] Anthropic. 2024. Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku. <https://www.anthropic.com/news/3-5-models-and-computer-use>.
- [8] Anthropic / Claude. 2025. Claude Code. <https://claude.com/product/claude-code>. Accessed: 2025-12-11.
- [9] AnySphere. 2024. Cursor: The AI Code Editor. <https://cursor.com>.
- [10] Fengyao Bai, Hongbin Zhang, Zhitao Chen, Jiangsu Du, Zhiguang Chen, and Yutong Lu. 2026. AlignedServe: Orchestrating Prefix-aware Batching to Build a High-throughput and Computing-efficient LLM Serving System. *Proceedings of the ACM on Management of Data* 4, 3 (2026), 1–25.
- [11] Soumya Basu, Aditya Sundarajan, Javad Ghaderi, Sanjay Shakkottai, and Ramesh Sitaraman. 2018. Adaptive TTL-based caching for content delivery. *IEEE/ACM transactions on networking* 26, 3 (2018), 1063–1077.
- [12] Shiyi Cao, Dacheng Li, Fangzhou Zhao, Shuo Yuan, Sumanth R. Hegde, Connor Chen, Charlie Ruan, Tyler Griggs, Shu Liu, Eric Tang, Richard Liaw, Philipp Moritz, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. SkyRL-Agent: Efficient RL Training for Multi-turn LLM Agent. *arXiv:2511.16108 [cs.AI]* <https://arxiv.org/abs/2511.16108>
- [13] Mingyue Cheng, Jie Ouyang, Shuo Yu, Ruiran Yan, Yucong Luo, Zirui Liu, Daoyu Wang, Qi Liu, and Enhong Chen. 2025. Agent-R1: Training Powerful LLM Agents with End-to-End Reinforcement Learning. *arXiv:2511.14460 [cs.CL]* <https://arxiv.org/abs/2511.14460>
- [14] Yihua Cheng, Yuhua Liu, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Kuntai Du, and Junchen Jiang. 2025. LMCACHE: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. *arXiv preprint arXiv:2510.09665* (2025).
- [15] Edith Cohen, Eran Halperin, and Haim Kaplan. 2005. Performance aspects of distributed caches using TTL-based consistency. *Theoretical computer science* 331, 1 (2005), 73–96.
- [16] Cursor. 2025. Agents | Cursor. <https://cursor.com/agents>. Accessed: 2025-12-11.
- [17] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *arXiv:2205.14135 [cs.LG]* <https://arxiv.org/abs/2205.14135>
- [18] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Attentionstore: Cost-effective attention reuse across multi-turn conversations in large language model serving. *arXiv preprint arXiv:2403.19708* 52 (2024), 20–38.
- [19] Shihong Gao, Xin Zhang, Yanyan Shen, and Lei Chen. 2025. Apt-serve: Adaptive request scheduling on hybrid cache for scalable llm inference serving. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–28.
- [20] In Gim, Zhiyao Ma, Seung-seob Lee, and Lin Zhong. 2025. Pie: A Programmable Serving System for Emerging LLM Applications. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (Lotte Hotel World, Seoul, Republic of Korea) (SOSP '25)*. Association for Computing Machinery, New York, NY, USA, 415–430. <https://doi.org/10.1145/3731569.3764814>
- [21] Hendri Hendri, Rukmi Sari Hartati, Linawati Linawati, and Dewa Made Wiharta. 2024. Optimizing CDN Modeling with API Integration Using Time ToLive (TTL) Caching Technique. *Jurnal Ekonomi Manajemen Sistem Informasi (JEMSI)* 6, 2 (2024).
- [22] Tomas Hernandez-Quintanilla, Eduardo Magaña, Daniel Morató, and Mikel Izal. 2021. On the reduction of authoritative DNS cache timeouts: Detection and implications for user privacy. *Journal of Network and Computer Applications* 176 (2021), 102941.
- [23] Carlos E. Jimenez, John Yang, Kilian Lieret, Alex L. Zhang, and Ofir Press. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? <https://github.com/SWE-bench/SWE-bench>.
- [24] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770* (2023).
- [25] Jaeyeon Jung, Arthur W. Berger, and Hari Balakrishnan. 2003. Modeling TTL-based Internet caches. In *IEEE INFOCOM 2003. Twenty-second Annual Joint Conference of the IEEE Computer and Communications Societies (IEEE Cat. No. 03CH37428)*, Vol. 1. IEEE, 417–426.
- [26] Hao Kang, Ziyang Li, Xinyu Yang, Wei Li, Yinfang Chen, Junxiong Wang, Beidi Chen, Tushar Krishna, Chenfeng Xu, and Simran Arora. 2026. ThunderAgent: A Simple, Fast and Program-Aware Agentic Inference System. *arXiv:2602.13692 [cs.OS]* <https://arxiv.org/abs/2602.13692>
- [27] Balachander Krishnamurthy, Craig Wills, and Yin Zhang. 2001. On the use and performance of content distribution networks. In *Proceedings of the 1st ACM SIGCOMM Workshop on Internet Measurement*. 169–182.
- [28] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.
- [29] LMCACHE Lab and vLLM. 2025. vLLM Production Stack. <https://docs.vllm.ai/projects/production-stack/en/latest/>
- [30] LangChain. 2025. ReAct-style agents —LangChain documentation. https://python.langchain.com/api_reference/langchain/agents/langchain.agents.react.base.ReActChain.html.
- [31] LangGraph. 2025. StateGraph and graph-based state machines —LangGraph. https://langchain-ai.github.io/langgraph/concepts/agentic_concepts/.
- [32] David Lawrence, Warren Kumari, and Puneet Sood. 2020. Serving stale data to improve DNS resiliency. (No Title) (2020).
- [33] Kilian Lieret, John Yang, Carlos E. Jimenez, Alexander Wettig, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2025. mini-SWE-agent: The 100-line AI agent that resolves GitHub issues on SWE-bench. <https://github.com/SWE-agent/mini-swe-agent>.
- [34] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. 2024. Parrot: Efficient serving of {LLM-based} applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 929–945.
- [35] Tim Lin. 2025. Overview of Function Calling in Open-Source Models. <https://medium.com/%40c22647809/overview-of-function-calling-in-open-source-models-cc23e9b13360>.
- [36] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. 2023. Agentbench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688* (2023).
- [37] Yuhua Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, et al. 2024. Cachegen: Kv cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 38–56.
- [38] Michael Luo, Naman Jain, Jaskirat Singh, Sijun Tan, Ameen Patel, Qingyang Wu, Alp Arayak, Colin Cai, Tarun Venkat, Shang Zhu, Ben Athiwaratkun, Manan Roongta, Ce Zhang, Li Erran Li, Raluca Ada Popa, Koushik Sen, and Ion Stoica. 2025. DeepSWE: Training a Fully Open-sourced, State-of-the-Art Coding Agent by Scaling RL. <https://www.together.ai/blog/deepswe>. Together AI blog post, July 2, 2025.
- [39] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E. Gonzalez, et al. 2025. Autellix: An efficient serving engine for llm agents as general programs. *arXiv preprint arXiv:2502.13965* (2025).
- [40] Huanzhi Mao, Raymond Tsao, Jingzhuo Zhou, Shishir G. Patil, and Joseph E. Gonzalez. 2025. BFCL V4: Web Search. https://gorilla.cs.berkeley.edu/blogs/15_bfcl_v4_web_search.html.
- [41] Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Jason Huang, Vishnu Suresh, Yixin Huang, Xiaowen Yu, Joseph E. Gonzalez, and Shishir G. Patil. 2024. BFCL V3 • Multi-Turn & Multi-Step Function Calling Evaluation. https://gorilla.cs.berkeley.edu/blogs/13_bfcl_v3_multi_turn.html.
- [42] Giovane CM Moura, John Heidemann, Ricardo de O Schmidt, and Wes Hardaker. 2019. Cache me if you can: Effects of DNS time-to-live. In *Proceedings of the Internet Measurement Conference*. 101–115.
- [43] Rajesh Nishtala, Hans Fugal, Steven Grimm, Marc Kwiatkowski, Herman Lee, Harry C. Li, Ryan McElroy, Mike Paleczny, Daniel Peek, Paul Saab, et al. 2013. Scaling memcache at facebook. In *10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13)*. 385–398.
- [44] Qwen. 2024. Function Calling —Qwen Documentation. https://qwen.readthedocs.io/en/latest/framework/function_call.html.
- [45] Shuo Ren, Pu Jian, Zhenjiang Ren, Chunlin Leng, Can Xie, and Jiajun Zhang. 2025. Towards scientific intelligence: A survey of llm-based scientific agents. *arXiv preprint arXiv:2503.24047* (2025).
- [46] Keshav Santhanam, Deepti Raghavan, Muhammad Shahir Rahman, Thejas Venkatesh, Neha Kunjal, Pratiksha Thaker, Philip Levis, and Matei Zaharia. 2024. Alto: An efficient network orchestrator for compound ai systems. In *Proceedings of the 4th Workshop on Machine Learning and Systems*. 117–125.

- [47] sgl project. [n.d.]. sglang. <https://github.com/sgl-project/sglang>. Accessed: 2025-12-09.
- [48] Vikranth Srivatsa, Zijian He, Reyna Abhyankar, Dongming Li, and Yiyang Zhang. 2024. Preble: Efficient Distributed Prompt Scheduling for LLM Serving. arXiv:2407.00023 [cs.DC] <https://arxiv.org/abs/2407.00023>
- [49] Xin Tan, Yimin Jiang, Yitao Yang, and Hong Xu. 2025. Towards end-to-end optimization of llm-based applications with ayo. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 1302–1316.
- [50] Noppanat Wadlom, Junyi Shen, and Yao Lu. 2026. Efficient LLM serving for agentic workflows: A data systems perspective. *Proceedings of the ACM on Management of Data* 4, 3 (2026), 1–29.
- [51] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. arXiv:2407.16741 [cs.SE] <https://arxiv.org/abs/2407.16741>
- [52] Xingyao Wang, Zihan Wang, Jiateng Liu, Yangyi Chen, Lifan Yuan, Hao Peng, and Heng Ji. 2023. Mint: Evaluating llms in multi-turn interaction with tools and language feedback. *arXiv preprint arXiv:2309.10691* (2023).
- [53] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. 2023. Fast distributed inference serving for large language models. *arXiv preprint arXiv:2305.05920* (2023).
- [54] Tian Xia, Ziming Mao, Jamison Kerney, Ethan J. Jackson, Zhifei Li, Jiarong Xing, Scott Shenker, and Ion Stoica. 2025. SkyWalker: A Locality-Aware Cross-Region Load Balancer for LLM Inference. arXiv:2505.24095 [cs.DC] <https://arxiv.org/abs/2505.24095>
- [55] Zhiqiang Xie. 2025. SGLang HiCache: Fast Hierarchical KV Caching with Your Favorite Storage Backends. <https://lmsys.org/blog/2025-09-10-sglang-hicache/>
- [56] Fanjia Yan, Huanzhi Mao, Charlie Cheng-Jie Ji, Ion Stoica, Joseph E. Gonzalez, Tianjun Zhang, and Shishir G. Patil. 2024. Berkeley Function-Calling Leaderboard. https://gorilla.cs.berkeley.edu/blogs/8_berkeley_function_calling_leaderboard.html
- [57] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. arXiv:2405.15793 [cs.SE] <https://arxiv.org/abs/2405.15793>
- [58] Juncheng Yang, Yao Yue, and KV Rashmi. 2021. A large-scale analysis of hundreds of in-memory key-value cache clusters at twitter. *ACM Transactions on Storage (TOS)* 17, 3 (2021), 1–35.
- [59] Jiayi Yao, Hanchen Li, Yuhua Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*. 94–109.
- [60] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045 [cs.AI] <https://arxiv.org/abs/2406.12045>
- [61] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*.
- [62] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. arXiv:2501.01005 [cs.DC] <https://arxiv.org/abs/2501.01005>
- [63] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 521–538.
- [64] Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Linhao Zhang, Shulin Xin, Lu Chen, Qi Liu, Xiaojian Zhong, Aoyan Li, Siyao Liu, Yongsheng Xiao, Liangqiang Chen, Yuyu Zhang, Jing Su, Tianyu Liu, Rui Long, Kai Shen, and Liang Xiang. 2025. Multi-SWE-bench: A Multilingual Benchmark for Issue Resolving. arXiv:2504.02605 [cs.SE] <https://arxiv.org/abs/2504.02605>
- [65] Wei Zhang, Zhiyu Wu, Yi Mu, Banruo Liu, Myungjin Lee, and Fan Lai. 2025. Tempo: Application-aware LLM Serving with Mixed SLO Requirements. *arXiv preprint arXiv:2504.20068* (2025).
- [66] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems* 37 (2024), 62557–62583.
- [67] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 193–210.
- [68] Kan Zhu, Yufei Gao, Yilong Zhao, Liangyu Zhao, Gefei Zuo, Yile Gu, Dedong Xie, Tian Tang, Qinyu Xu, Zihao Ye, Keisuke Kamahori, Chien-Yu Lin, Ziren Wang, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. 2025. NanoFlow: Towards Optimal Large Language Model Serving Throughput. arXiv:2408.12757 [cs.DC] <https://arxiv.org/abs/2408.12757>